

kickstart_lecture4_matplotlib

December 19, 2024

0.1 Python Kickstart Lecture 4: Matplotlib for Plotting

```
[1]: # Import libraries: Numpy and Matplotlib

import numpy as np
import matplotlib.pyplot as plt
```

Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.

Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.

0.1.1 Line Plot

```
[2]: # Generate random x-y Data

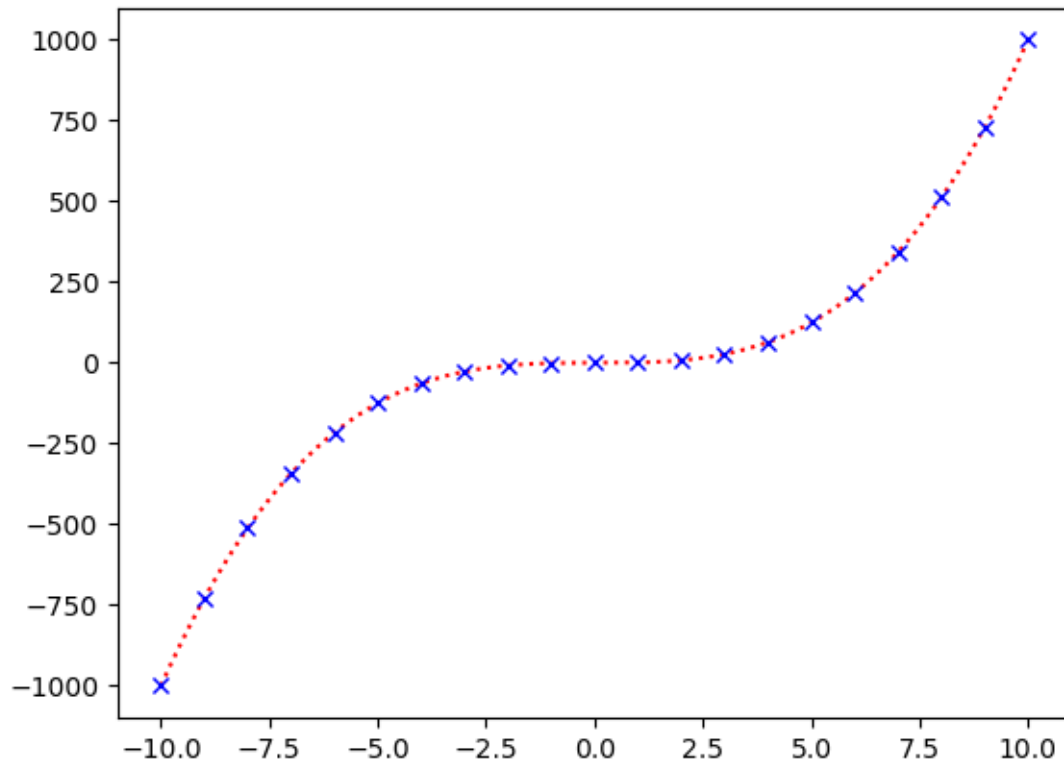
x = np.linspace(-10,10,201)
y = x**3-1
```

```
[3]: x2 = np.linspace(-10,10,21)
y2 = x2**3-1
```

```
[9]: # Create plot

plt.figure()
plt.plot(x,y, ':', color = 'red')
plt.plot(x2,y2, 'x',color = 'blue')
```

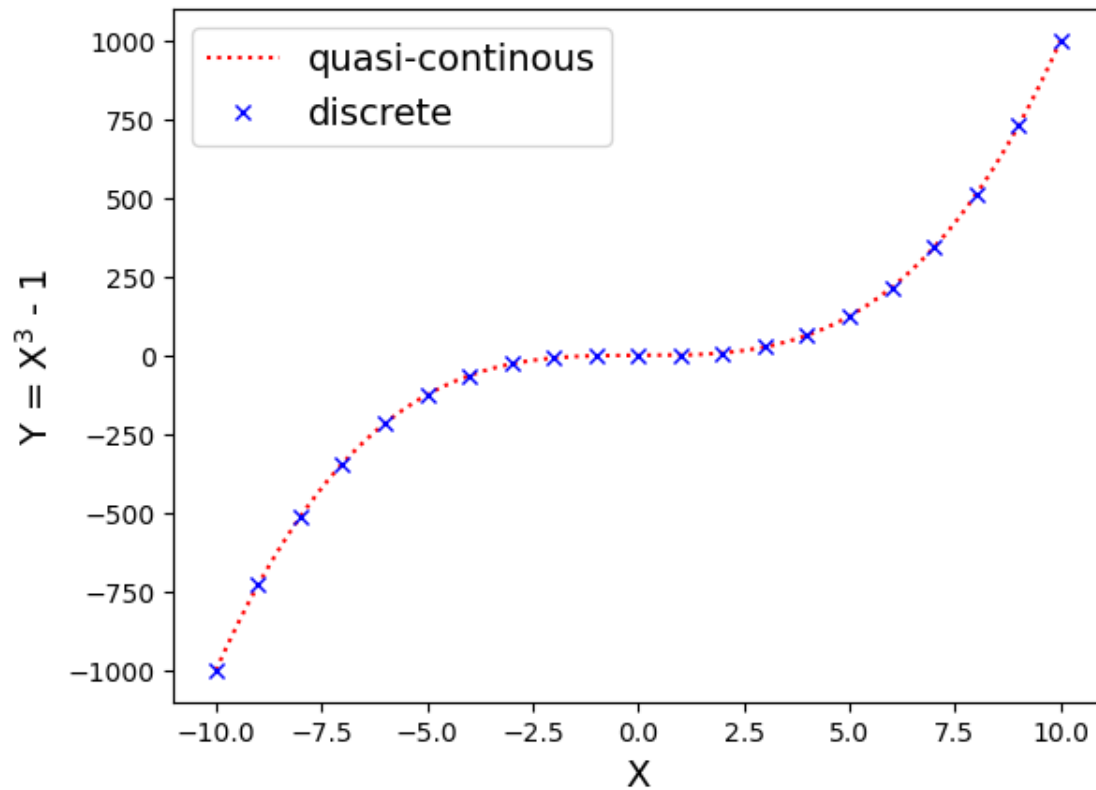
```
[9]: [<matplotlib.lines.Line2D at 0x11700ee70>]
```



```
[14]: # Add axis labels, legends, fontsize

plt.figure()
plt.plot(x,y, 'r:', label = 'quasi-continuous')
plt.plot(x2,y2, 'bx',label = 'discrete')
plt.legend(fontsize = 14)
plt.xlabel('X', fontsize = 14)
plt.ylabel('Y = X3 - 1', fontsize = 14)
```

```
[14]: Text(0, 0.5, 'Y = X3 - 1')
```



```
[ ]: # Adjust color, linestyle -> see above
```

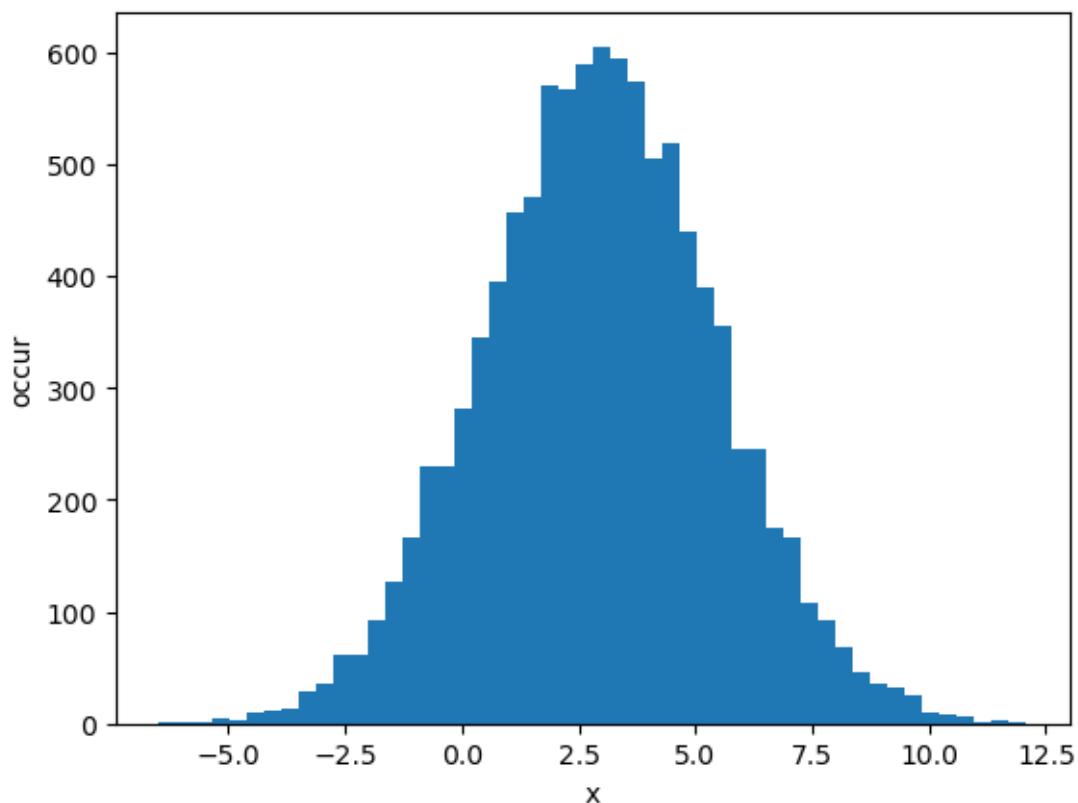
0.1.2 Histogram

```
[26]: # Generate gaussian distributed data
```

```
gaussian_data = 3 + 2.5 * np.random.randn(10000)
```

```
[27]: plt.figure()
plt.hist(gaussian_data, 50)
plt.xlabel('x')
plt.ylabel('occur')
```

```
[27]: Text(0, 0.5, 'occur')
```



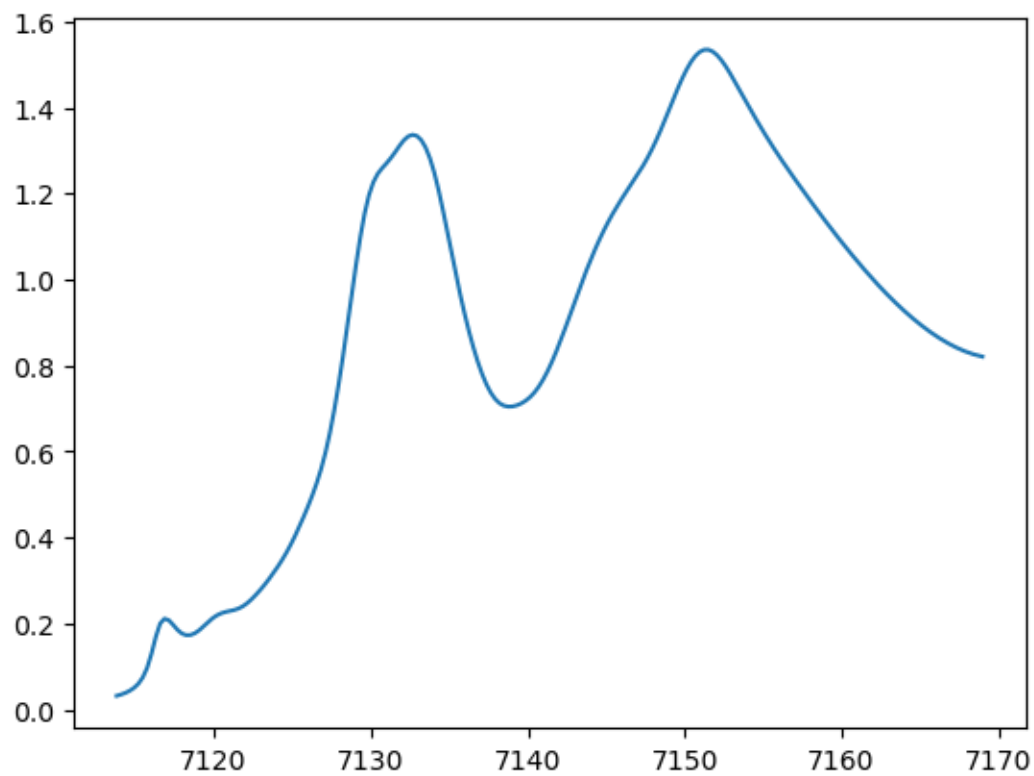
0.1.3 Loading Data and Visualizing

```
[36]: # Load XAS data with loadtxt

data = np.loadtxt('xas_FeC05_from_materialsproject.tsv',
                  delimiter='\t',
                  skiprows = 4,
                  usecols = [0,1]
                  )
x_data = data[:,0]
y_data = data[:,1]
```

```
[39]: # Create a plot
plt.figure()
plt.plot(x_data,y_data)

plt.savefig('Fe_xas.png', dpi = 300) # pixelated
plt.savefig('Fe_xas.eps') # vectorized
```



```
[38]: # And save the plot to png and eps
```

<Figure size 640x480 with 0 Axes>

```
[ ]:
```